

A Dynamic-Precision Bit-Serial Computing Hardware Accelerator for Solving Partial Differential Equations Using Finite Difference Method

Junjie Mu¹, Graduate Student Member, IEEE, and Bongjin Kim², Senior Member, IEEE

Abstract—This article presents an all-digital hardware accelerator for solving partial differential equations using the finite difference method (FDM) with dynamically reconfigurable computing bit precision. The proposed accelerator consists of 21×21 bit-serial processing elements (PEs) to compute 2-D grid solutions with massive parallelism. The 21×21 bit-serial PEs are connected in a lattice structure, and a PE communicates with four neighboring PEs to update the grid solutions. A PE comprises four key building blocks: a bit-serial adder, a shift register, 4:1 multiplexers, and an accumulator. The proposed hardware accelerator minimizes data movement based on its array architecture that directly maps a 2-D grid of the FDM. Besides, the proposed residue-based bit-serial computation method lowers energy consumption and latency. The checkerboard update method further improves the performance by updating the solutions in two cycles regardless of the grid size. A test chip is fabricated using 65 nm, and a 21×21 PE array occupies 0.462 mm^2 . The measured energy consumption is 1.59 nJ per iteration at 16 bit, 1 V, and 25.6 MHz.

Index Terms—Bit-serial computing, checkerboard-shape grid update, dynamic precision, finite difference method (FDM), graph accelerator, partial differential equations (PDEs), residue-based.

I. INTRODUCTION

NUMERICAL analysis finds applications in broad scientific and engineering fields. Approximation of solutions to the partial differential equations (PDEs) has already been used in various applications, such as weather forecasting, the prediction of economic trends, and the simulation of physical phenomena [1], [2]. While analytical methods can provide the exact solutions with no error, it is rare to find such solutions. Instead, the numerical methods can approximate the solutions with minimal errors.

Various numerical methods have been developed, including the finite difference method (FDM) [3], [4], the finite

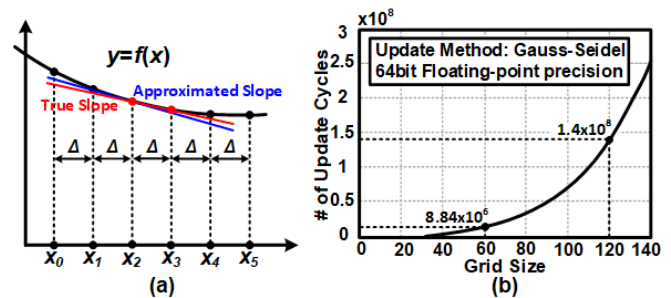


Fig. 1. (a) Concept of FDM. (b) Number of update cycles required for different grid sizes when solving a 2-D PDE using FDM.

element method (FEM) [5], and the finite volume method (FVM) [6]. The FDM has become one of the most common methods for finding numerical solutions to PDEs based on its ease of implementation. A PDE is converted into a set of linear algebraic equations that can be computed using conventional computers. When solving PDEs using FDM, the domain (spatial or temporal) of the problem is discretized into a grid, as shown in Fig. 1(a). The grid solution is then approximated by solving the linear equations containing the finite differences and the solutions from neighboring points. While the approximation is computed by solving a set of simple linear equations, it requires many iterations until it converges to a solution with high accuracy. Besides, the number of iterations increases quadratically as the discrete grid size increases.

Various hardware accelerators have been developed to solve PDEs using FDM more efficiently than the traditional general-purpose hardware. Hardware platforms [7], [8] based on GPU and field-programmable gate array (FPGA) have been developed for solving PDEs, which provide the benefits of flexibility and programmability. The GPU-based accelerators can solve a more comprehensive class of problems. However, they face the challenge of high energy consumption due to massive data communication between computing units and memories. The computationally expensive FDM consumes $\sim 15 \text{ mJ}$ per iteration when solving a 2-D Poisson equation with 128×128 grids on GPU [9]. FPGA-based solvers could reduce latency and energy consumption, but high energy consumption has yet to be resolved due to massive memory access. Hence, the development of ASIC accelerators with

Manuscript received 9 December 2021; revised 28 March 2022 and 21 April 2022; accepted 6 May 2022. Date of publication 20 May 2022; date of current version 30 January 2023. This article was approved by Associate Editor Vivek De. (Corresponding author: Bongjin Kim.)

Junjie Mu is with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798.

Bongjin Kim is with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA 93106 USA (e-mail: bongjin@ucsb.edu).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSSC.2022.3174354>.

Digital Object Identifier 10.1109/JSSC.2022.3174354

0018-9200 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

specific computing models is necessary to improve latency, throughput, and energy consumption.

Recently, hardware accelerators based on digital [10], analog [11], [12], hybrid [13]–[15], and in-memory computing (IMC) [9], [16] methods have been proposed to solve PDEs. Kung *et al.* [10] presented a programmable digital accelerator architecture for solving dynamic ordinary and partial differential equations using a multilayer cellular nonlinear network (CeNN) model. The accelerator based on memory-central architecture can solve diverse differential equations and improve energy efficiency by several orders of magnitudes. Analog accelerators [11], [12] show superiority in solving nonlinear differential equations based on their natural implementations of nonlinear functions, reducing the workload of the digital solvers. However, they have suffered from low computing precision and analog circuit non-idealities such as nonlinearity and noise susceptibility. Besides, solving high-dimensional equations using analog accelerators increases the area overhead and complicates the signal routings. A hybrid (analog/digital) accelerator [13] is proposed to solve differential equations up to fourth orders using continuous-time approximation. The computation accuracy has been improved by using an 8-bit precision with a digitally assisted calibration. Nevertheless, the issues, including the limited precision (8 bit) and the large area overhead from analog circuits, have not been addressed yet.

IMC accelerators based on SRAM/RRAM [9], [16] have been implemented to solve PDEs using FDM. An SRAM-based PDE solver [9] implemented a residue-based FDM computation in the SRAM array and lowered computing precision to 5 bit. A multigrid method with a hybrid layer update method is used to speed up the convergence. The current-based analog computation in memory achieves high energy efficiency and throughput, but it suffers from poor linearity and accuracy due to analog non-idealities, such as memory cell variations and noise susceptibilities. An RRAM-based IMC solver [16] finds approximate PDE solutions with FDM using a vector-matrix multiplication in the memory array storing sparse coefficient matrices. Non-zero coefficient matrices are stored in multiple RRAM macros and merged to overcome the precision limitations of the immature RRAM technologies. However, the multi-macro merging method introduces extra peripheral circuit overhead and generates additional quantization errors.

This article presents a graph hardware accelerator for solving PDEs with FDM using a 2-D bit-serial processing element (PE) array. The PE array enables massive parallelism and reduces the overall energy consumption by embedding a bit-serial computing unit in each PE in the grid, localizing FDM operation and minimizing data communication. The proposed accelerator's performance and energy efficiency have been significantly improved by adopting a checkerboard grid update method and dynamically changing the precision while updating the solutions using residues.

This article significantly expands on the previous publication [22] by discussing related algorithms, detailed circuit designs, potential improvement scenarios, and more measurement results. This article is organized as follows. Section II introduces the algorithms for solving PDEs and

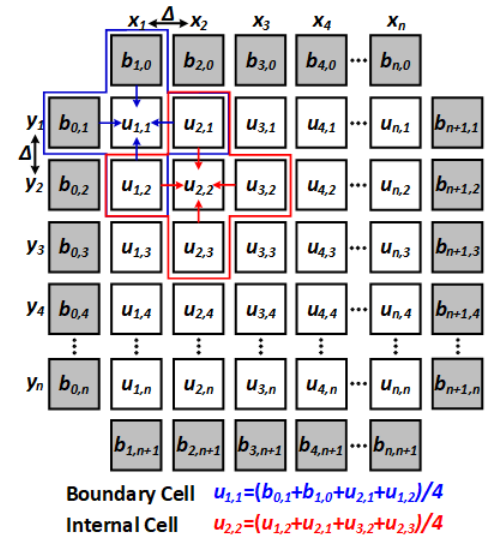


Fig. 2. 2-D grid for FDM with boundary conditions.

improving their convergence. Section III presents the hardware implementation, including the overall architecture and the residue-based bit-serial PE circuits with timing diagrams. Section IV discusses the remaining challenges and the scalability of the proposed PDE accelerator. Measurement results are summarized in Section V, and finally, Section VI concludes this article.

II. ALGORITHMS FOR SOLVING PDES

A. Finite Difference Method

FDM is one of the most mature approaches in numerical analysis. It approximates derivatives with finite differences based on the Taylor series [3], which is the sum of an infinite number of derivatives of the function at a certain point. The 2-D Laplace equation, expressed in (1), is used as an example to explain the FDM algorithm

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0. \quad (1)$$

A 2-D plane is discretized into an $N \times N$ grid with step sizes Δx and Δy along the x - and the y -axis, as shown in Fig. 2. The nodes in the 2-D grid represent solutions ($u_{i,j}$), and boundary conditions are applied right next to the grid on four sides (top, bottom, left, and right). The derivatives of x and y are approximated with the horizontal and vertical nearest values based on FDM. Assuming that a distance (Δ) between two adjacent points along the x - and the y -axis is equal, the second derivatives of x and y are expressed as follows:

$$\frac{\partial^2 u_{i,j}}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta^2} \quad (2)$$

$$\frac{\partial^2 u_{i,j}}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta^2}. \quad (3)$$

The approximated second derivatives of x and y , (2) and (3), are substituted into the Laplace equation in (1), and hence, the solution $u_{i,j}$ is approximated to the average of the four solution

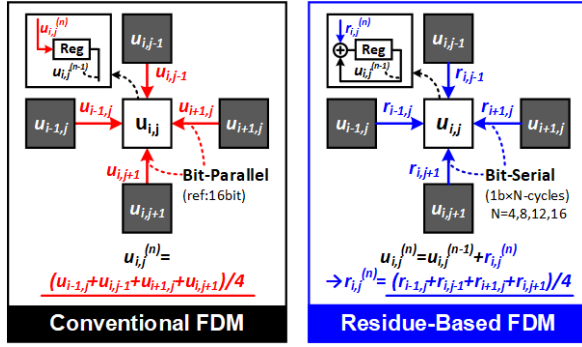


Fig. 3. Conventional bit-parallel computing (left) and the proposed residue-based bit-serial computing for solving PDEs using FDM.

values from neighbors ($u_{i-1,j}$, $u_{i+1,j}$, $u_{i,j-1}$, and $u_{i,j+1}$) as follows:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i+1,j} + u_{i-1,j} - 4u_{i,j} + u_{i,j+1} + u_{i,j-1}}{\Delta^2} \quad (4)$$

$$u_{i,j} = \frac{1}{4} [u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}]. \quad (5)$$

All values in the $N \times N$ grid are updated based on (5) until the solutions are converged to the steady-state values.

B. Residue-Based Bit-Serial Computing

The conventional FDM-based approximation using fixed bit-precision computing hardware requires significant overhead in energy consumption due to memory access. Instead of having a separate memory to read/write updated solutions for the 2-D grid of FDM, we propose a graph hardware accelerator that directly maps the FDM grid to a 2-D PE array, where each PE represents a pixel in the grid. As a result, we save significant energy by performing data communication only between neighboring PEs.

In addition to the graph hardware accelerator architecture, we further improved the energy efficiency by adopting a bit-serial computing method. The bit-serial computing not only reduces computation circuit complexity but also significantly relaxes the requirement for communication channels between PEs. The required bit width of a channel between PEs is 1 bit regardless of the used bit precision (e.g., 16/32/64 bit) for computing FDM. It also adds flexibility in terms of the achievable bit precision as the bit-serial computing can easily update its bit precision by changing the number of cycles to represent a word.

In this work, we propose a residue-based FDM, as shown in Fig. 3, to fully utilize the advantages of bit-serial computing by dynamically updating the number of cycles per operation based on the residue magnitude while updating the 2-D grid solutions. The residue-based bit-serial computing method reduces energy consumption and computing latency compared to traditional fixed precision computing. In residue-based FDM, the residues ($r_{i,j}$) are used to update the 2-D grid solution ($u_{i,j}$) by accumulating them, differentiating from the conventional FDM that simply overwrites the grid solutions by averaging their neighbors. When using residue-based FDM

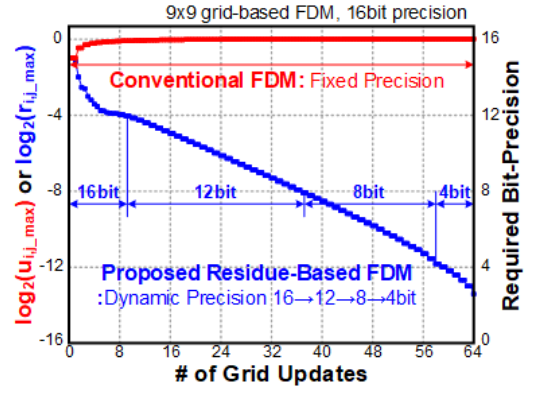


Fig. 4. Comparison of the conventional FDM with fixed precision and the proposed residue-based FDM with dynamic precision.

to solve 2-D Laplace equations, the boundary conditions are applied only in the first iteration and the initial solution ($u_{i,j}^{(0)}$) is zero stored in each PE. In the following iterations, the boundary conditions are zero, and residues are updated according to the following formula (6). The updated residues are accumulated until the residue ($r_{i,j}^{(n)}$) are 0 and solution ($u_{i,j}^{(n)}$) converges:

$$r_{i,j}^{(n)} = \frac{1}{4} [r_{i+1,j} + r_{i-1,j} + r_{i,j+1} + r_{i,j-1}] \quad (6)$$

$$u_{i,j}^{(n)} = r_{i,j}^{(1)} + r_{i,j}^{(2)} + \dots + r_{i,j}^{(n)} = u_{i,j}^{(n-1)} + r_{i,j}^{(n)}. \quad (7)$$

Fig. 4 compares the simulated results showing the magnitude of the data used for communication (i.e., $u_{i,j}$ for the conventional FDM and $r_{i,j}$ for the residue-based FDM) for a 9×9 grid size. In this simulation, all the boundary conditions are set to ones, and the initial 2-D grid solutions are zeros. The residue continuously decreases as the grid solutions are getting close to their steady-state values, enabling the reduction of the required bit precision from 16 to 4 bit.

C. Checkerboard Update Method

The choice of the grid solution update method plays a crucial role in achieving high energy efficiency and throughput while finding numerical PDE solutions using FDM, which requires a larger number of iterations for updating the grid solutions until they converge.

The Jacobi method [17] is a standard algorithm, which updates all the grid nodes based on the solution from the previous iteration (i.e., $(n-1)$ th iteration) as follows:

$$u_{i,j}^{(n)} = \frac{1}{4} [u_{i+1,j}^{(n-1)} + u_{i-1,j}^{(n-1)} + u_{i,j+1}^{(n-1)} + u_{i,j-1}^{(n-1)}]. \quad (8)$$

Even though some of the neighboring grid solutions have been already updated, we only use their solutions from the previous iteration for updating the next grid point. Hence, the Jacobi method can achieve the best parallelism as all grid points can be updated in parallel as long as their solutions from the previous iterations are accessible, as shown in Fig. 5 (top). However, the convergence speed is low since the grid solutions

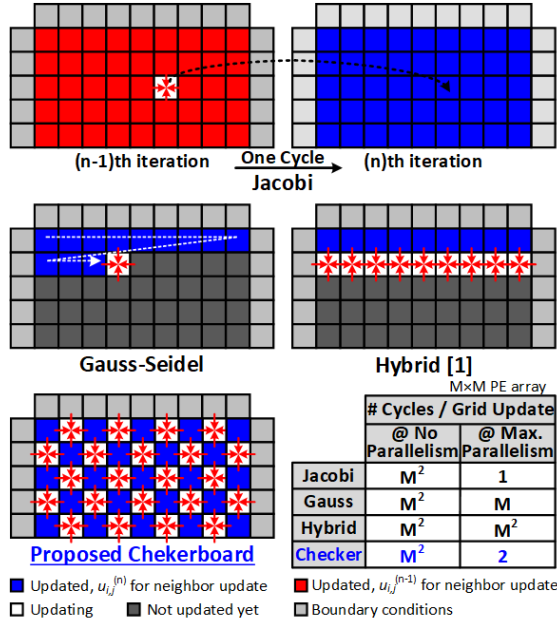


Fig. 5. Three existing (i.e., Jacobi, Gauss-Seidel, and hybrid) and the proposed checkerboard-shape update method.

can only update after the current iterations for all the grid points are completed.

Unlike the Jacobi method, the Gauss-Seidel method [18] updates its grid solutions based on the latest available solutions from their neighbor grid points, as shown in the following equation:

$$u_{i,j}^{(n)} = \frac{1}{4} \left[u_{i+1,j}^{(n-1)} + u_{i-1,j}^{(n-1)} + u_{i,j+1}^{(n-1)} + u_{i,j-1}^{(n-1)} \right]. \quad (9)$$

The newest grid solutions are used for updating the subsequent grid solutions, and therefore, the convergence speed is faster than the Jacobi method. However, it could degrade the throughput due to the limited parallelism. The hybrid method is used [9] to balance convergence speed and parallelism. The grid is divided into multiple layers, and the values in one layer are updated in parallel, as shown in Fig. 5 (top right). The updated solutions in the current layer are immediately used for the next layer update.

In this work, we adopt a checkerboard-shape update method [19] to optimize both throughput and convergence speed. A grid is partitioned into two groups forming the checkerboard shape, and the grid solutions in each group are updated in parallel, as shown in Fig. 5 (bottom right). While achieving massive parallelism (i.e., 50% of the Jacobi method), the proposed checkerboard method achieves the fastest convergence as the latest grid solutions are used for updating the grid solutions for each cycle of checkerboard-shape solution updates.

The FDM energy and computing latency with four different types of update methods are quantified based on a 9×9 PE array, as shown in Fig. 6. The Jacobi method consumes the largest energy (i.e., the largest number of PE runs) to achieve the mean absolute error (MAE) of 10^{-3} , while the Gauss-Seidel and checkerboard methods require the least energy to

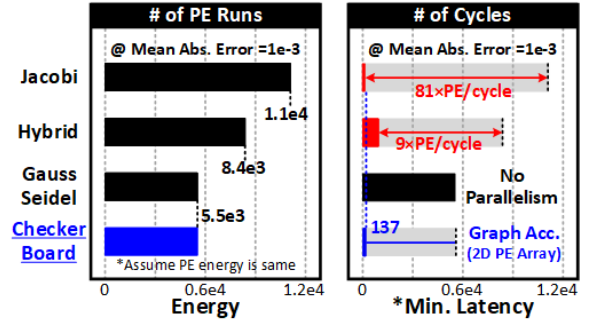


Fig. 6. Energy and latency comparisons between the existing and the proposed update methods for solving PDEs using FDM.

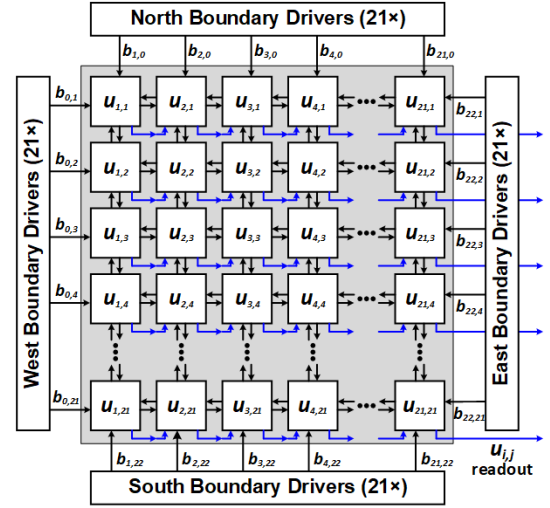


Fig. 7. Overall architecture of the proposed PDE solver test chip.

achieve the same error level. The latency is the smallest for the Jacobi and the checkerboard methods by assuming maximum parallelism for all methods.

III. HARDWARE IMPLEMENTATION

Fig. 7 shows the overall architecture of the proposed 2-D PDE solving hardware accelerator test chip, comprising 21×21 PEs connected in a lattice graph structure and four boundary condition drivers. Residues and solutions are updated in the PE array, while off-chip boundary conditions are transmitted to the PE array through four boundary drivers. Fig. 8 shows a detailed block diagram of PE, comprising four circuit building blocks: a bit-serial adder, a shift register (i.e., a divide-by-4 and serial-to-parallel converter), 4:1 multiplexers (MUXs), and an accumulator.

A. PE Circuit

The bit-serial adder comprises three half adders (in the first column), nine full adders, and four D-flip-flops (DFFs). The top four adders combine the residues from the four neighbors bit by bit in a serial manner. The carry-out signals from the adders are stored in the four DFFs and transferred back to adders as carry-in signals in the next cycle for the bit-serial addition. The sums are passed to the next adders on the right

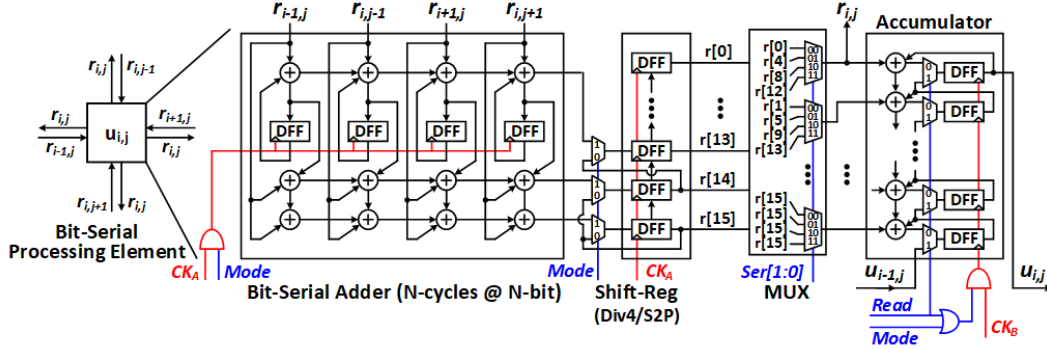


Fig. 8. Block diagram of the proposed bit-serial PE with bit-serial adder, shift register, MUX, and accumulator.

and then added and stored in the shift register. The bottom eight adders are used for adding carry-out signals at the MSB cycle of bit-serial addition for the sign extension. Note that the bit-serial addition circuit is reused for N cycles regardless of the bit precision, resulting in a minimal hardware overhead for high-precision PDE solving.

A 16-bit shift register with three MUXs shifts the bit-serial adder outputs from the bottom to the top, while the bit-serial addition is performed. It also serves as a divide-by-4 circuit and a serial-to-parallel converter for the accumulation to update the solution ($u_{i,j}$). The two least significant bits are shifted out by shifting the bit-serial adder outputs to the top of the 16 DFFs in series, resulting in the equivalent divide-by-4 operation without extra circuitry. Note that the bit-serial adder and the divide-by-4 circuit operate when the *Mode* signal is 1. When the *Mode* signal is 0, they are reused for serializing the residue from the previous operation cycle and transmitting them bit-by-bit to its four neighboring PEs.

On the right of the shift register, $16 \times$ parallel 4:1 MUXs route the computed residue to the accumulator based on the bit precision mode selection (00: 16 bit, 01: 12 bit, 10: 8 bit, and 11: 4 bit). A 16-bit accumulator on the right of the parallel 4:1 MUXs updates and stores the PE solution by accumulating the updated residues (i.e., averaged using incoming residues from the four neighboring PEs) at each operation cycle. The accumulator is reused when reading out the solution of the PEs. When the *Read* signal is 1, the circuit configuration is updated to a shift register and used to shift the stored PE solution bits to the right to read out the solutions.

B. Dynamic-Precision Implementation

Fig. 9 describes the implementation of the proposed dynamic precision in the residue-based FDM. The bit-serial adder takes N cycles to perform accumulation. The generated outputs of the adders (i.e., 3 bit) are stored in three DFFs located at the bottom of the shift register circuit and then shifted to the top. After the bit-serial computing is completed, $16 \times$ parallel 4:1 MUXs dynamically route the shift register output to the accumulator. For instance, the outputs of shift registers are connected to the first inputs of the corresponding DFFs of the accumulator via 4:1 MUXs when *Ser*[1:0] is 00 at

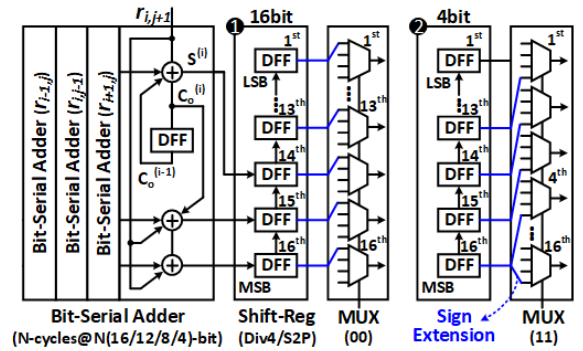


Fig. 9. Implementation of dynamic precision at 16 and 4 bit.

16-bit mode. At the 12-bit precision mode (i.e., when *Ser*[1:0] is 01), the 5th to the 16th DFF outputs (from the top) in the shift register are connected to the 1st to the 12th DFF of the accumulator. The 16th DFF output of the shift register is also connected to the 13th to the 16th DFF of the accumulator for the sign extension. Similarly, the 8-bit (or 4 bit) precision mode connects their 9th to the 16th DFF outputs (or 13th to the 16th DFF outputs) in the shift register to the 1st to the 8th DFF (or 1st to the 4th DFF) of the accumulator, while the final DFF outputs from the shift registers are used for their sign extensions.

C. Timing Diagram

Fig. 10 shows the operation timing diagram for 8- and 4-bit precision modes. A reset (*Rst*) signal is enabled to clear all initial data, and the bit precision selection signal (i.e., *Ser*[1:0]) is set to 10 to enable the 8-bit precision mode. Four residues (i.e., $r_{i-1,j}$, $r_{i,j-1}$, $r_{i+1,j}$, and $r_{i,j+1}$) from the neighbors are 39, -25, 13, and 51, respectively. In the odd cycle, it takes eight bit-serial operation cycles (CK_A) to accumulate four 8-bit serial residue inputs from the neighboring PEs. Meanwhile, the accumulated residues are divided by four based on the shift register operation. The averaged residue (i.e., 19) is then accumulated into the solution ($u_{i,j}$), which is enabled by a cycle of CK_B . The updated residue is serialized from LSB to MSB in the even cycle. It is then transmitted to the

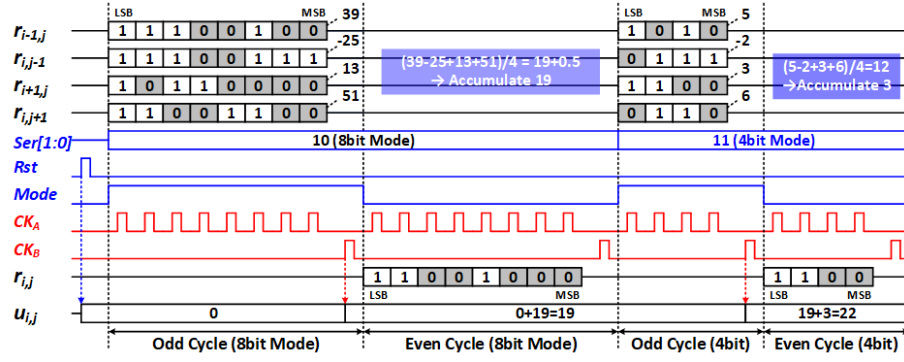


Fig. 10. Operation timing diagram of bit-serial PE in even/odd cycles at 8-/4-bit mode.

neighboring PEs for their bit-serial addition and divide-by-4 operations.

Note that the *Mode* signal determines whether the PE is under an even or odd operation cycle. For instance, the PE computes bit-serial addition and divide-by-4 when the *Mode* signal is 1, while it serializes its residue when the *Mode* signal is 0. The *Ser[1:0]* signal switches from 10 to 11 to update the precision mode from 8 to 4 bit. After the precision is updated to 4 bit, the only noticeable difference is the reduced number of CK_A cycles (from 8 to 4) for bit-serial computation.

IV. DISCUSSION

A. Applications

The Laplace equation can describe time-independent equilibrium conditions, such as electrostatics, heat conduction, and fluid dynamics. The Laplace equation is the steady-state heat equation in heat conduction problems. When the fluid is incompressible and irrotational potential flows, the velocity potential satisfies the classical Laplace equation (i.e., Laplace irrotational motion problem.) The Laplace equation is encountered in electrostatics (in the absence of a charge density) and gravity when the solutions represent electric potential and gravitational potential. Besides, the Laplace equation can be used for image processing, for instance, image denoising and edge detection. In this article, the proposed solver is evaluated in three applications, including heat conduction, electrostatics, and potential dynamic.

B. MAE and Delta-Sigma Modulation

The impact of grid size and bit precision on MAE is explored in Fig. 11. Ideally, the MAE decreases continuously as the number of operation cycles (for grid update) increases with no quantization error. However, finite precision computing results in quantization errors, resulting in limitations in the minimum level of achievable MAE. As shown in Fig. 11, the simulated MAE decreases as the precision increases from 16 to 32 bit. Besides, the grid size also affects the MAE (i.e., the greater the grid size, the higher the MAE in the saturation). The larger grid (20×20) slows down the time it takes to converge to the saturated MAE levels, and the saturated MAE is higher than the smaller grid (10×10). A residue-based FDM has been introduced in this work to reduce computing

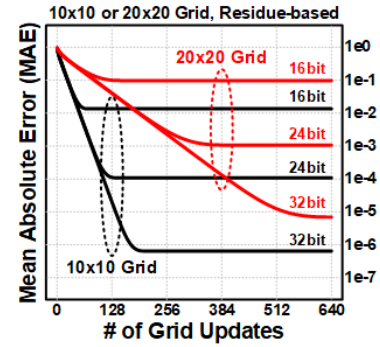


Fig. 11. MAE versus grid size and bit precision.

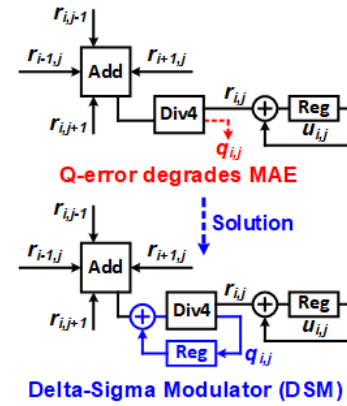


Fig. 12. First-order delta-sigma modulation to improve the degraded MAEs due to quantization errors and residue-based FDM methods.

latency and energy compared to the conventional approach by dynamically adjusting the precision of the residue based on its magnitude. However, the proposed residue-based method creates extra quantization error that could degrade the overall accuracy.

We can insert a first-order delta-sigma modulator (DSM), as shown in Fig. 12, to improve the MAE that has been degraded due to quantization errors caused by finite bit precisions and residue-based FDM. The first-order DSM circuit reuses the quantization error ($q_{i,j}$) generated by each cycle of bit-serial residue computation by adding the error together

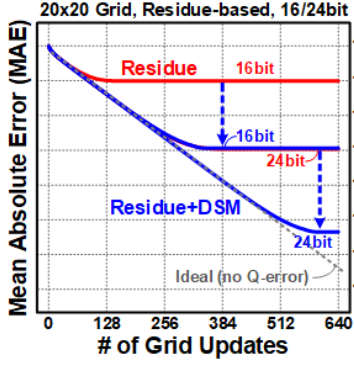


Fig. 13. MAE w/ and w/o DSM.

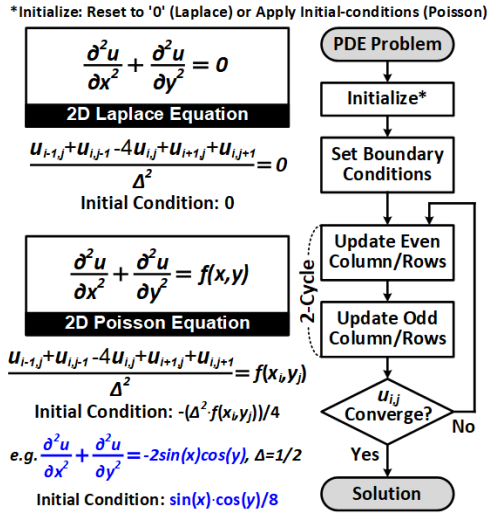


Fig. 14. Flowchart (right) for solving Laplace/Poisson equations (left).

with the new residue value at the subsequent operation cycles. Fig. 13 shows the simulated MAEs with and without using DSM with fixed 16- and 24-bit precisions. Note that the gray dotted line shows the ideal MAE trend with no quantization error. It is observed that the MAE of the residue-based FDM has been significantly improved by two orders of magnitude with DSM for both 16- and 24-bit precisions.

C. Solving Poisson Equations

The proposed PDE solver can also find numerical solutions for Poisson equations. Fig. 14 (right) shows a flowchart for solving 2-D Laplace and Poisson equations. First, the initial conditions associated with step sizes and source functions are defined by discretizing the PDEs using FDM and programmed as the initial residue values. Note that the initial conditions are 0 for Laplace equations and $-(\Delta^2 f(x_i, y_j))/4$ for Poisson equations. The boundary conditions are applied in the first two cycles and then reset to 0. After the first two cycles, the boundary conditions applied to four sides of the 2-D FDM grid propagate through the PE array and update their residues and solutions. It takes two operation cycles to update the whole grid, and we iterate this until the solution $(u_{i,j})$ converges.

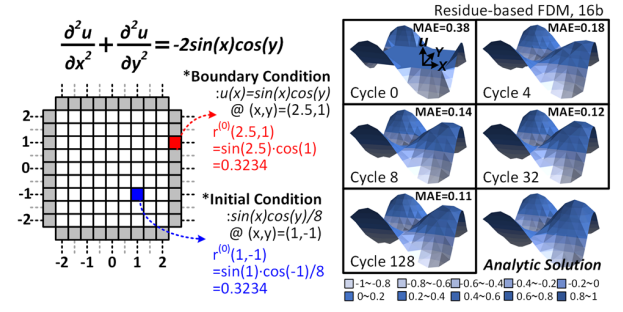
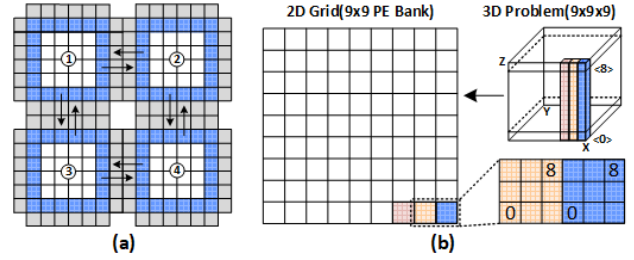
Fig. 15. Example of solving a 2-D Poisson equation using the proposed residue-based PDE solver architecture with a 9×9 FDM grid.Fig. 16. (a) Multicore scalability. (b) Mapping of 3-D problem with a grid size of $9 \times 9 \times 9$ into a 2-D PDE solver with 9×9 PEs.

Fig. 15 shows an example of solving a 2-D Poisson equation using the proposed PDE solver architecture with a 9×9 FDM grid using initial conditions (i.e., $\sin(x)\cos(y)/8$) and boundary conditions [i.e., $\sin(x)\cos(y)$]. Fig. 15 (left) shows the mapping of boundary conditions and initial conditions on the 9×9 grid, and Fig. 15 (right) shows transient maps of the 2-D FDM grids, showing the convergence behavior as we iterate the operations.

D. Multi-Core/Chip and 3-D Scalability

The proposed PDE solver applications are not restricted to the problems on a 21×21 grid (i.e., PE array size). For finding numerical solutions for the PDEs with the larger FDM grid size, we can consider building a multi-core/chip configuration. Fig. 16(a) shows an example of a four-core connection. While the PEs located in non-boundary regions (i.e., boundary PEs are located at the top and the bottom rows, and the leftmost and the right-most columns of the PE array) interact with their neighboring PEs in the same core, boundary PEs are required to interact with external PEs from other cores or chips, as shown in Fig. 16(a). Therefore, we need high-speed communication links, requiring significant energy and area overheads. Note that the bit-serial computation is beneficial in multi-core/chip implementations as it significantly reduces the required number of communication channels.

Besides the scalability of the proposed hardware for solving the larger 2-D PDEs, we can also reconfigure the proposed PE array for solving 3-D PDEs. For instance, we can map a 3-D grid into a 2-D PE array by grouping some of the PEs to represent a column of the 3-D grid, as shown in Fig. 16(b). An example of 3-D PDEs with a $9 \times 9 \times 9$ grid size can be

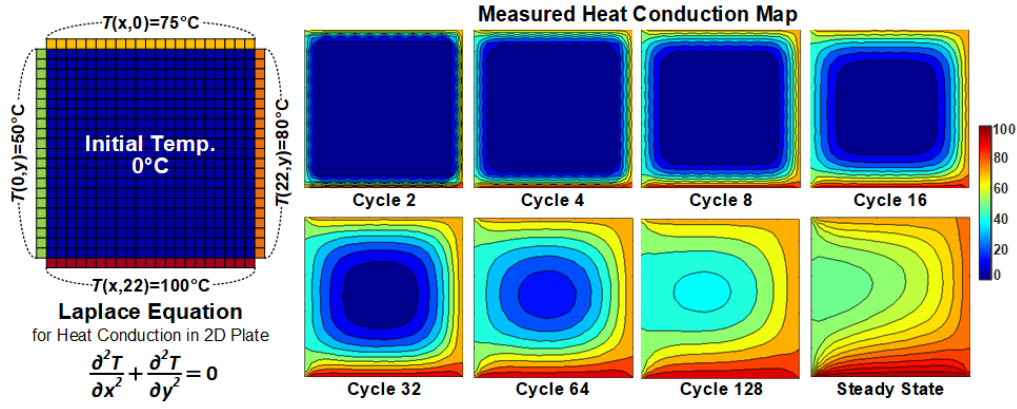


Fig. 17. Measured numeral PDE solution versus update cycle for a Laplace equation of heat conduction in the 2-D plate.

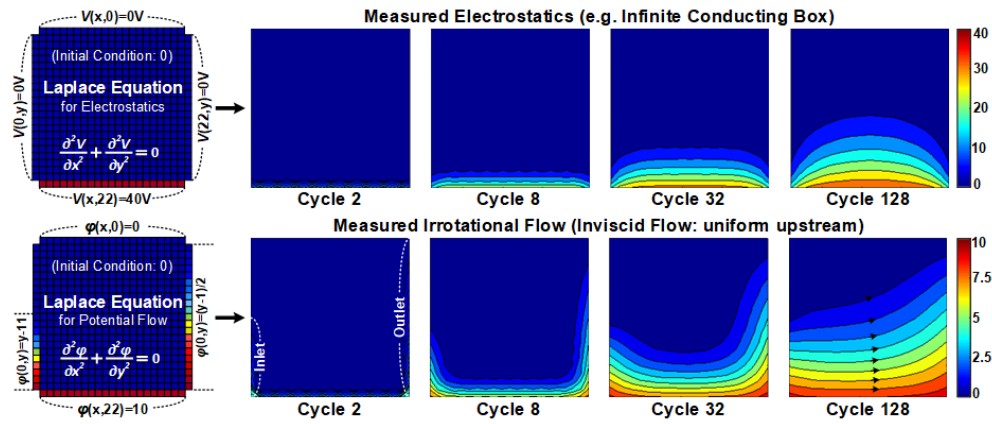


Fig. 18. Measured numeral PDE solutions for electrostatics and potential flow.

mapped to a 2-D 9×9 PE group, where each group consists of 3×3 PEs. A 3×3 PE group represents a column in the 3-D grid. This placement can reduce the routing overhead compared to a strip placement (i.e., 9 PE cells arranged in the same column or row) [20].

V. MEASUREMENT RESULTS

A. Practical Application

Fig. 17 shows the measured numerical solutions of the heat conduction in a 2-D plate. The 2-D heat conduction problem can be expressed as a Laplace equation, and boundary conditions are only required to find its numerical solutions (i.e., stabilized temperature after heat conduction) using FDM. The boundary conditions are set to fixed temperatures (i.e., Dirichlet boundary conditions), 50 °C, 75 °C, 80 °C, and 100 °C, as shown in Fig. 17. The proposed residue-based FDM combined with the checkerboard update method requires boundary conditions to be alternatively applied only for the first two operation cycles. After that, the boundary conditions are set to 0 °C. The boundary conditions applied at the first two cycles are gradually propagated through the 2-D PE array and used for updating the temperatures for all the 2-D grid pixels. Fig. 17 (right) shows the measured transient heat conduction

maps from the 2nd to the 284th cycle, where the temperature is settled to a steady state.

Fig. 18 (top) shows the measured electrostatics, where the electric potentials change in a 2-D infinite conducting box with zero charge density. The electric potential change with space follows a 2-D Laplace equation, where the boundary conditions are fixed voltages (i.e., the bottom side is 40 V and the other three sides are 0 V), as shown in Fig. 18 (top left). It took 164 operation cycles to reach a steady state. Fig. 18 (bottom) shows the measured incompressible and irrotational potential flow. The boundary conditions are fixed, as shown in Fig. 18 (bottom left), and the steady state was reached after 203 operation cycles.

B. Power and Energy Consumption

The measured power consumption of PE at 16-bit precision with a range of operating frequencies (0.4–25.6 MHz) and core supply voltages (0.6–1.2 V) is shown in Fig. 19. A PE consumes 8.3 μW (or 1.88 μW) at 1.2-V supply (or 0.6 V) at 16-bit precision and 25.6 MHz. Fig. 20 shows the measured energy consumption per PE update with different bit precisions, supply voltages, and operating frequencies. As shown in Fig. 20(a), the PE energy consumption varies from 69 fJ (at 0.6 V) to 339.8 fJ (at 1.2 V) at 1.6 MHz and from 73.3 fJ

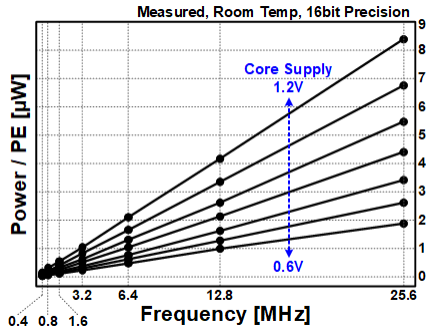


Fig. 19. Measured power versus frequency.

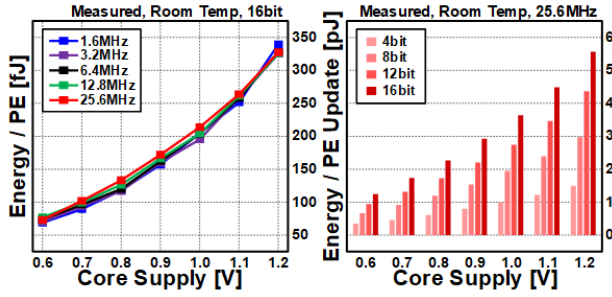


Fig. 20. Measured energy consumption.

(at 0.6 V) to 327.7 fJ (at 1.2 V) at 25.6 MHz when the precision is fixed to 16 bit. The measured energy increases as the bit precision changes from 4 to 16 bit, as shown in Fig. 20 (right), due to the increased number of clock cycles for serializing the residue to transmit and computing bit-serial addition for updating solutions.

C. Area and Power Breakdown

Fig. 21 (top) shows the power and area breakdown of the proposed PDE solver. The bit-serial PE array and the peripheral circuits occupy 98.6% and 1.4% of the total core area, respectively. The computing circuits for residue occupy 58.4% of the total area, and 40.2% is dedicated to the circuitry for updating solutions. Similarly, 97.8% of the power is consumed by the PEs, where 46.9% and 50.9% of them are used to update residues and solutions, respectively. The peripheral circuitry consumes the remaining 2.2% of the power.

Fig. 22 plots the area/power breakdown of combinational and sequential logic in each PE. The combinational logic occupies 19.27% of the total PE area. The area overhead of sequential logic is 80.73%. The combinational logic consumes 21.6% of the power, and 78.4% of the power is consumed by the sequential logic.

D. Performance and Comparison

Fig. 21 (bottom) shows a performance summary table of the proposed PDE solver. The PDE solver consists of 21×21 PEs, where each PE is capable of performing bit-serial addition with 4–16 bit (4/8/12/16 bit) dynamic precision. A compact 65-nm bit-serial PE occupies a $32.4 \times 32.4 \mu\text{m}^2$ die area, resulting in

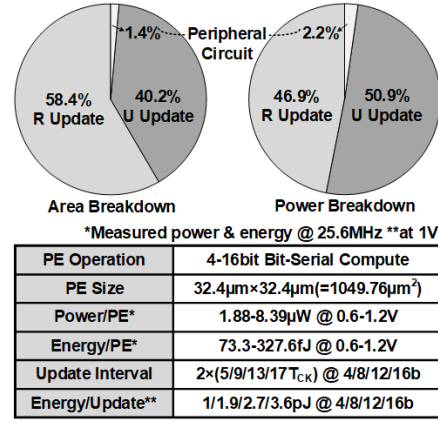


Fig. 21. Area/power breakdown and PE summary table.

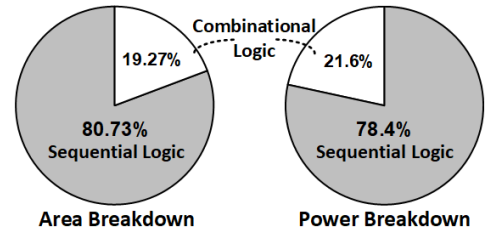


Fig. 22. PE area/power breakdown of combinational and sequential logic.

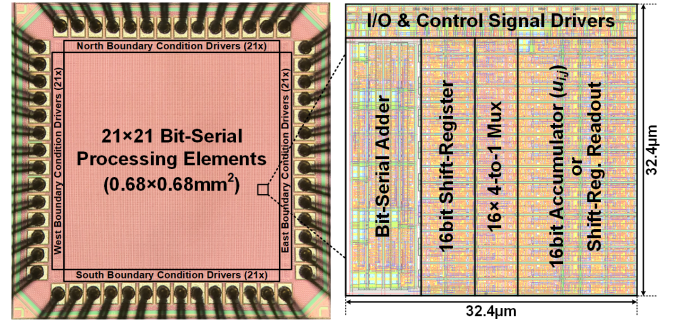


Fig. 23. 65-nm test-chip die micrograph (left) and PE layout.

a 21×21 PE array (with the core area of $0.68 \times 0.68 \text{ mm}^2$) integrated into a $1 \times 1 \text{ mm}^2$ test chip, including I/O PADs. The number of clock cycles for updating the solutions and the energy per each update is also summarized in the table of Fig. 21 (bottom).

Table I shows the comparison with the state-of-the-art PDE hardware accelerators using various process technologies and grid update methods using different types of computing circuits. To make a fair comparison among different technology nodes, we use the scaling scheme in [23], which shows the scaling formulations from 180- to 7-nm technologies. After scaling to the same technology and voltage, our proposed work presents the $1.82\times$ and $11\times$ power reduction compared with [9] and [12]. We also compare the solution energy with GPU and FPGA by estimating the energy consumption through [24] and [25]. When using these approaches to reach the same accuracy as ours, our proposed solver consumes

TABLE I
PERFORMANCE COMPARISON WITH STATE-OF-THE-ART PDE SOLVING ACCELERATORS

	JSSC 2016 [14]	Nature Elec. 2018 [16]	JSSC 2020 [9]	TVLSI 2021 [21]	JSSC 2021[12]	This Work
Technology	65nm CMOS	Memristor Xbar	180nm CMOS	65nm Model	180nm CMOS	65nm CMOS
Application	Approximate Computing	PDE Solving	PDE Solving	PDE Solving	1-D Wave Equation	PDE Solving
Update Method	N/A	Jacobi Method	Hybrid Method	N/A	N/A	Checkerboard
Grid Update Latency (@M×M Grid)	N/A	M ² Cycles (Cell-by-Cell)	M Cycles (Row-by-Row)	N/A	N/A	2 Cycles (Even/Odd)
Analog/Digital	Hybrid	Analog	Analog	Analog	Analog	Digital
Computing Type	Continuous-Time Hybrid	In-Memory Computing	In-Memory Computing	In-Memory Computing	Finite-difference Time-domain	Bit-Serial Computing
Calibration	Required	Required	Required	Required	Not Required	Not Required
Graph Accelerator (# of 2D-Grid PEs)	No	No	No	No	No	Yes (21×21 PEs)
Core Supply	1.2V	1.25V	1.8V	1.8V	2.2V	0.6-1.2V
Die Area	3.8mm ²	N/A	11mm ²	N/A	11.26mm ²	1.0mm ²
Core Area	2.0mm ²	N/A	^D 1.868mm ²	N/A	N/A	0.462mm ²
Computation Precision	Fixed 8bit	Fixed 8bit	Fixed 5bit	Fixed 11bit	^B Fixed 7.9Bit	Dynamic 4/8/12/16bit
Power Consumption @ Frequency/Supply (^E Scaled Power)	1.2mW @ ^A 20kHz/1.2V	N/A	66.4mW @200MHz/1.8V (4.39mW)	N/A	560mW @ ^C 0.25MHz/2.2V (26.5mW)	2.41mW @25.6MHz/1V

^A 20kHz is the unity gain frequency of integrator and is mapped to physical frequency for solving time-varying PDEs based on time scaling factor ^B ENOB=7.9 ^C Compute bandwidth with ADC ^D Core area only includes the 4 MAC macros ^E Scaled power consumption from 180nm to 65nm @1V

142× and 9× less total energy compared to GPU and FPGA, respectively. The proposed work is the first fully digital implementation of the graph-based hardware accelerator performing massively parallel computations for solving PDEs using FDM compared to prior works. Fig. 23 shows a die micrograph of the 65-nm test chip and the proposed bit-serial PE layout.

VI. CONCLUSION

Solving PDEs with numerical methods such as FDM requires high-precision and massive iterative computations that are time- and energy-consuming. In this work, a graph-based PDE solver with residue-based FDM and the checkerboard grid update method is proposed to improve both performance and energy efficiency. The bit-serial computing method has minimized the hardware overhead in building a 2-D PE array for solving PDEs based on FDM with massive parallelism. The accelerator based on a lattice graph architecture minimizes data movement, improving energy efficiency and throughput than conventional von Neumann architecture. A residue-based dynamic precision further enhances energy efficiency and reduces latency. A test chip implementing a 21 × 21 bit-serial PE array has been implemented using a 65-nm LP CMOS process, and it demonstrated solving 2-D Laplace equations. The architecture is readily scalable and solves large-scale 2-D/3-D PDEs, including Laplace and Poisson equations. The measured energy of PE for a PDE update is 1 pJ at 4 bit and 3.6 pJ at 16-bit precision with 1-V supply voltage and 25.6-MHz operating frequency.

REFERENCES

- [1] P. Bauer, A. Thorpe, and G. Brunet, "The quiet revolution of numerical weather prediction," *Nature*, vol. 525, no. 7567, pp. 47–55, Sep. 2015.
- [2] Y. Achdou, F. J. Buera, J.-M. Lasry, P.-L. Lions, and B. Moll, "Partial differential equation models in macroeconomics," *Phil. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 372, no. 2028, Nov. 2014, Art. no. 20130397.
- [3] G. Mur, "Absorbing boundary conditions for the finite-difference approximation of the time-domain electromagnetic-field equations," *IEEE Trans. Electromagn. Compat.*, vol. EMC-23, no. 4, pp. 377–382, Nov. 1981.
- [4] Acceleware. (2018). *Finite-Difference Time-Domain Solver and SDK*. [Online]. Available: <http://www.acceleware.com/fdtd-solvers>
- [5] S. R. Idelsohn and E. Oñate, "Finite volumes and finite elements: Two 'good friends,'" *Int. J. Numer. Methods Eng.*, vol. 37, no. 19, pp. 3323–3341, Oct. 1994.
- [6] H. Huang and A. Usmani, "Finite element method," in *Finite Element Analysis for Heat Transfer*, 1st ed. London, U.K.: Springer, 1994, pp. 21–47.
- [7] J. Krüger and R. Westermann, "Linear algebra operators for GPU implementation of numerical algorithms," *ACM Trans. Graph.*, vol. 22, no. 3, pp. 908–916, Jul. 2003.
- [8] O. Storaasli, "Computing faster without CPUs: Scientific applications on a reconfigurable FPGA-based hyper computer," in *Proc. 6th Mil. Aerosp. Program. Log. Devices Conf. (MAPLD)*, 2003.
- [9] T. Chen, J. Botimer, T. Chou, and Z. Zhang, "A 1.87-mm² 56.9-GOPS accelerator for solving partial differential equations," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1709–1718, Jun. 2020.
- [10] J. Kung *et al.*, "A programmable hardware accelerator for simulating dynamic systems," in *Proc. Int. Symp. Comput. Archit. (ISCA)*. Toronto, ON, Canada, June. 2017, pp. 403–415.
- [11] N. Udayanga *et al.*, "A radio frequency analog computer for computational electromagnetics," *IEEE J. Solid-State Circuits*, vol. 56, no. 2, pp. 440–454, Feb. 2021.
- [12] J. Liang, N. Udayanga, A. Madanayake, S. I. Hariharan, and S. Mandal, "An offset-cancelling discrete-time analog computer for solving 1-D wave equations," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2881–2894, Sep. 2021.
- [13] Y. Huang, N. Guo, M. Seok, Y. Tsividis, K. Mandli, and S. Sethumadhavan, "Hybrid analog-digital solution of nonlinear partial differential equations," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2017, pp. 665–678.
- [14] N. Guo *et al.*, "Energy-efficient hybrid analog/digital approximate computation in continuous time," *IEEE J. Solid-State Circuits*, vol. 51, no. 7, pp. 1514–1524, Jul. 2016.
- [15] G. E. R. Cowan, R. C. Melville, and Y. Tsividis, "A VLSI analog computer/digital computer accelerator," *IEEE J. Solid-State Circuits*, vol. 41, no. 1, pp. 42–53, Jan. 2006.
- [16] M. A. Zidan *et al.*, "A general memristor-based partial differential equation solver," *Nature Electron.*, vol. 1, pp. 411–420, Jul. 2018.
- [17] A.-K. C. Ahamed and F. Magoulès, "Efficient implementation of Jacobi iterative method for large sparse linear systems on graphic processing units," *J. Supercomput.*, vol. 73, no. 8, pp. 3411–3432, Aug. 2017.
- [18] M. Usui, H. Niki, and T. Kohno, "Adaptive Gauss–Seidel method for linear systems," *Int. J. Comput. Math.*, vol. 51, nos. 1–2, pp. 119–125, 1994.
- [19] M. ElMaghrbay, R. Ammar, and S. Rajasekaran, "Fast GPU algorithms for implementing the red-black Gauss–Seidel method for solving partial differential equations," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2013, pp. 269–274.

- [20] L. R. Everson, J. Song, S. S. Sapatnekar, and C. H. Kim, "A shortest path finding time-based accelerator core with built-in gravity control and buffer zone for smooth 3-D navigation," *IEEE Solid-State Circuits Lett.*, vol. 3, pp. 66–69, 2020.
- [21] S. S. Ensan and S. Ghosh, "ReLOPE: Resistive RAM-based linear first-order partial differential equation solver," *IEEE Trans. Very Large-Scale Integr. (VLSI) Syst.*, vol. 29, no. 1, pp. 234–241, Jan. 2021.
- [22] J. Mu and B. Kim, "A 21×21 dynamic-precision bit-serial computing graph accelerator for solving partial differential equations using finite difference method," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 406–407.
- [23] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm," *Integration*, vol. 58, pp. 74–81, Jun. 2017.
- [24] S. W. Keckler, W. J. Dally, B. Khailany, M. Garland, and D. Glasco, "GPUs and the future of parallel computing," *IEEE Micro*, vol. 31, no. 5, pp. 7–17, Sep. 2011.
- [25] B. Asgari, R. Hadidi, T. Krishna, H. Kim, and S. Yalamanchili, "Efficiently solving partial differential equations in a partially reconfigurable specialized hardware," *IEEE Trans. Comput.*, vol. 70, no. 4, pp. 524–538, Apr. 2021.



Junjie Mu (Graduate Student Member, IEEE) received the B.E. degree from the College of Information Science and Engineering, Northeastern University, Shenyang, China, in 2017, and the M.S. degree in electronics from Nanyang Technological University, Singapore, in 2018, where she is currently pursuing the Ph.D. degree with the School of Electrical and Electronic Engineering.

Her research interests include hardware accelerators for solving mathematical problems and memory-centric circuits.



Bongjin Kim (Senior Member, IEEE) received the B.S. and M.S. degrees from POSTECH, Pohang, South Korea, in 2004 and 2006, respectively, and the Ph.D. degree from the University of Minnesota, Minneapolis, MN, USA, in 2014.

He was with Rambus, Sunnyvale, CA, USA, where he was a Senior Staff Member and worked on the research of high-speed serial link circuits and microarchitectures. He was a Post-Doctoral Research Fellow with Stanford University, Stanford, CA, USA. From 2006 to 2010, he was with Samsung

Electronics, Yongin, South Korea, where he performed research on clock generators for high-speed serial links and clock generators. He also worked as a Research Intern with Texas Instruments, Dallas, TX, USA; IBM TJ Watson Research, Yorktown Heights, NY, USA; and Rambus from 2012 to 2014. He was an Assistant Professor with Nanyang Technological University, Singapore, from 2017 to 2020. He is currently an Assistant Professor with the Department of Electrical and Computer Engineering (ECE), University of California at Santa Barbara, Santa Barbara, CA, USA. His current research interests include quantum-inspired and brain-inspired computing; memory-centric computing devices, circuits, and architectures; hardware accelerators; alternative computing; and mixed-signal circuit design techniques and methodologies.

Dr. Kim has been serving as a Technical Program and Review Committee Member for Design Automation Conference (DAC), IEEE Custom Integrated Circuits Conference (CICC), and IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS) and a Guest Editor for the IEEE JOURNAL ON EMERGING AND SELECTED TOPICS IN CIRCUITS AND SYSTEMS (JETCAS). He was a recipient of the Prestigious Doctoral Dissertation Fellowship Award based on his Ph.D. research, the International Symposium on Low Power Electronics and Design (ISLPED) International Low Power Design Contest Award, and the Intel/IBM/Catalyst Foundation Award from CICC. His research works appeared at top peer-reviewed integrated circuit design and automation conference proceedings and journals, including IEEE International Solid State Circuits Conference (ISSCC), VLSI Symposium, IEEE JOURNAL OF SOLID-STATE CIRCUITS (JSSC), IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, CICC, European Solid-State Circuits Conference (ESSCIRC), IEEE Asian Solid-State Circuits Conference (ASSCC), ISLPED, Design, Automation and Test in Europe Conference (DATE), International Conference on Computer-Aided Design (ICCAD), and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMS (TVLSI).